Game Design Document

Revision

Version	Changes	Date
v1.0	First Verson of GDD	04/11/2025

Working Title: Echoes of the Depths
- **Game Overview**
- **Core Concept**
- **Gameplay Overview**
- **Game Loop**

I. Rules
- **Core Rules**
- **Progression Rules**
- **Combat & Puzzle Rules**

II. Player
- Overview
- Core Systems
- Attributes
- Controls
- Movement System
  - Movement Hierarchy
  - Technical Parameters
- Combat System
  - Attack Behavior
- Abilities
- Damage & Death Handling
- Implementation Notes
- Technical Considerations
- Future Additions

III. Enemies
- Overview
- **Enemy Types**
- **Bosses**

Behavior System
Core Components
Implementation Notes
Example: **Melee Enemy**
Behavior Summary
Behavior Tree Logic
Behavior Tree Diagram
Future Enemy Types
Technical Considerations
IV. World
Overview
World Structure
Exploration Goals
Draft Layout
V. Architecture
Overview
Folder Structure
Scene Management
Core Systems and Classes
ScriptableObjects
Save & Checkpoint System
VI. Arts
Art Direction
Environmental Samples
Character Design Direction

Working Title: Echoes of the Depths

Game Overview

Title: *Echoes of the Depths*

Genre: 2D Metroidvania / Action-Adventure

Engine: Unity

Platform: PC (Windows)

Art Style: Dark fantasy pixel art (using o-Lobster's "Another Metroidvania Asset Pack Vol. 1")

Core Gameplay: Exploration-based progression with a puzzle focus — players can advance by defeating bosses, with an option to make them easier by solving related puzzles.

Target Audience: Fans of 2D Metroidvania or action-platformers in general (*Hollow Knight*, *Castlevania, Metroid* or even *Mario, Kirby*).

Story Overview:

Long ago, the kingdom of Varelia thrived under a radiant crystal that pulsed with life. When the crystal fractured, a shadow seeped through the depths, corrupting all living beings.

You play as The Wanderer, an unnamed warrior revived by echoes of the crystal's energy. Guided by faint memories and spectral whispers, you descend into the forgotten ruins beneath the kingdom - confronting the echoes of its fallen guardians to restore the light or succumb to the depths.

Core Concept

Echoes of the Depths is a 2D exploration and progression-based game set in a ruined kingdom corrupted by otherworldly energy.

The player controls a warrior who explores a ruined castle to defeat the evil boss and restore the kingdom's balance.

Each region can be completed through combat-oriented encounters, with an optional puzzle-platform trials to make the boss encounters easier.

Design Goals:

- Create a connected world with meaningful backtracking.
- Balance combat and puzzle-solving.
- Keep systems simple but polished enough to showcase strong core mechanics.

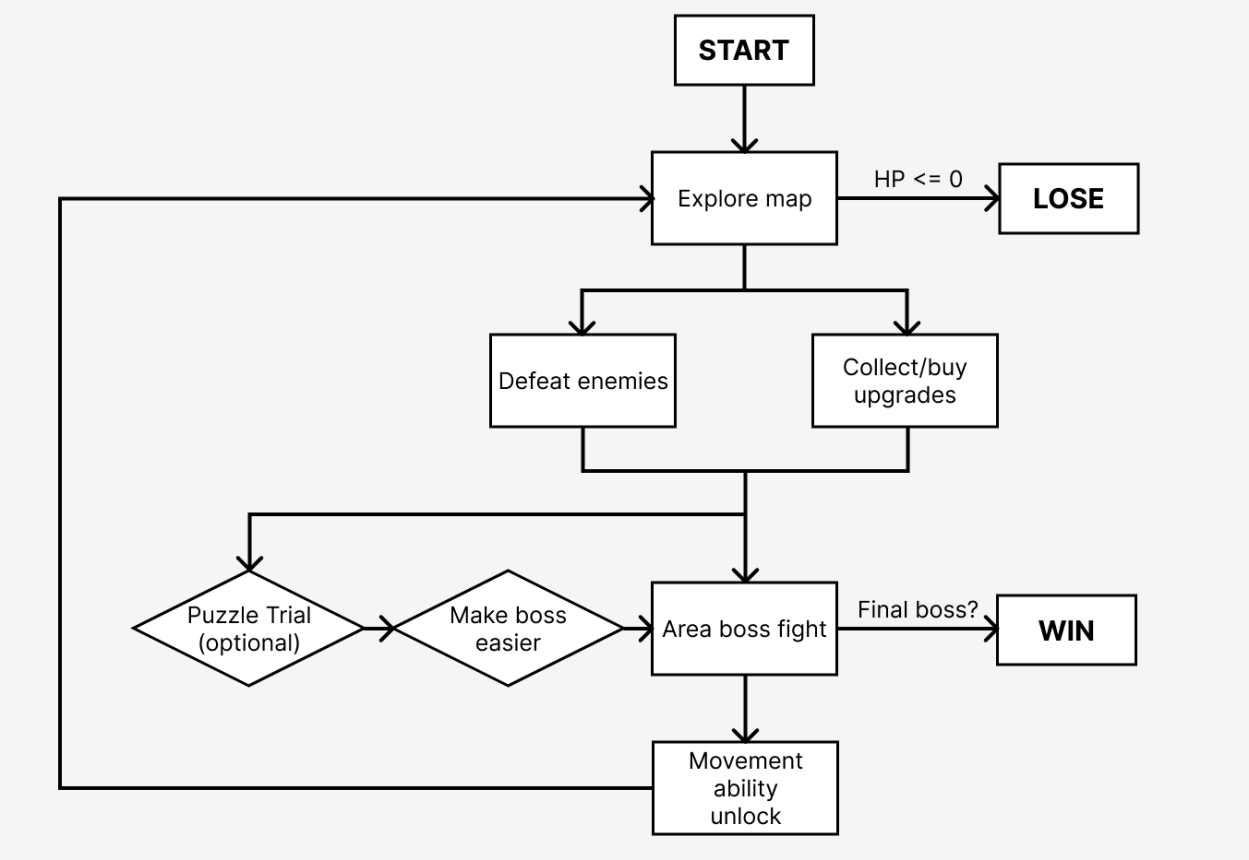
Gameplay Overview

- **Perspective:** 2D side-scrolling with platforming and combat.

- **Core Loop:** Explore → Fight or Solve → Unlock ability → Backtrack → Progress
- **Movement:** Run, jump, crouch, dash, wall climb, double jump.
- **Combat:** Melee-based with short combos and a dodge mechanic.
- **Puzzle-Platforming:** Uses environment switches, moving platforms, and ability-based traversal challenges.
- **Abilities:** Permanent upgrades that open new paths (e.g., Dash, Double Jump, Wall Climb, Ranged Pulse).
- **World Layout:**
  - Interconnected regions with multiple maps per area.
  - Central hub connecting all zones (maybe).
  - Each region provides one major ability unlock through either a boss or puzzle trial.
- **Progression:** Non-linear, ability gated areas.
- **Demo Goal (Week 12):** 1 fully playable area with 1 new ability unlock and a boss and puzzle trial option.

Game Loop

https://www.figma.com/design/4DBR99F3sAIlbCqy4J29c4/Echoes-of-the-Depths---Game-Loop-Diagram?node-id=3-89&t=6MiVtWVF59qodI5A-1



I. Rules

Core Rules

1. The player explores interconnected 2D regions using movement and combat abilities.
2. Progression is ability-based - each ability grants progress to newer areas.
3. A Puzzle Trial in the same region can be completed first to weaken or simplify the boss (e.g., disable attack patterns, reduce boss HP).
4. Defeating a boss grants a new ability that is tied to progression: double jump, e.g. dash, wall climb, super dash.
5. Checkpoints act as save points, restore health/energy, and reset enemy spawns.

6. Player death returns the character to the last checkpoint; all progress except defeated bosses and ability unlocks resets.
7. The player wins after defeating the final boss.

Progression Rules

- Each major region is designed around the latest unlocked ability, with some requiring a combination of abilities to get past certain platforming challenges.
- Puzzle trial completion makes the region's boss easier.
- Players can revisit previous areas (backtrack) to discover shortcuts, collectibles, and upgrades.
- Hidden routes and optional trials reward permanent stat or resource boosts.
- Some gates require multiple abilities to pass, ensuring gradual world expansion.

Combat & Puzzle Rules

Combat

- Standard enemies respawn after rest or area reload.
- Bosses have unique attack patterns and arena mechanics.
- Completing the puzzle before the fight weakens the boss or alters the arena in the player's favor.

Puzzle

- Trials test platforming precision and ability usage (switches, timing, traversal).
- Failure respawns the player at the nearest checkpoint without penalties.

II. Player

Overview

The **Player** is the central agent of *Echoes of the Depths*, embodying fluid motion, responsive combat, and ability-based progression.

The design goal is to make the player feel agile and precise — capable of chaining movement techniques seamlessly between exploration and battle.

All player logic is governed by a **modular component system**, with separate scripts handling movement, combat, and abilities.

The architecture prioritizes readability and expandability, making it easy to add new abilities or tweak core parameters without refactoring.

Core Systems

System	Description	Key Class
PlayerController	Handles movement, jumping, dashing, and physics behaviors. Integrates Unity's Input System.	`PlayerController.cs`
PlayerCombat	Manages melee attacks, attack range detection, and enemy hit logic.	Integrated into `PlayerController`
AbilityManager	Tracks and activates unlocked abilities (Dash, Double Jump, Wall Jump, Pulse, Super Dash).	`AbilityManager.cs`
GameManager	Stores player state (HP, unlocked abilities, respawn point) and handles death/respawn.	`GameManager.cs`
UIManager	Displays health, damage flash, and fade transitions.	`UIManager.cs`

Attributes

Attribute	Description
Health (HP)	Player dies at 0 HP. Restores at checkpoints. Affected by relics or collectibles.
Energy (EP)	Consumed for energy-based actions (Ranged Pulse, Super Dash). Regenerates over time or via combat.
Attack Power	Determines melee damage; increased through upgrades or relics.

Attribute	Description
Defense	Mitigates incoming damage.
Movement Speed	Base run velocity, affected by gear or ability modifiers.

Controls

Player input uses the **Unity Input System** with dedicated action maps.

Each action maps to a corresponding InputAction in `PlayerController.cs`.

Action	Default Binding	Function
Move	`Left/Right` or Left Stick	Horizontal movement.
Jump	`Space` or A/Cross	Jump; performs Double Jump if mid-air.
Dash	`Left Shift` or RB/R1	Quick burst movement; disabled during cooldown.
Attack	`Z` or Right Bumper (RB)	Performs melee strike.
Pulse (Ability)	`A` or Left Trigger (LT)	Fires short-range energy pulse (once unlocked).
Pause / Menu	`Esc` or Start	Toggles pause menu.

Each input triggers a callback in the PlayerController (e.g. `OnJumpPerformed`, `OnDashPerformed`), which routes to the correct system behavior.

Movement System

The PlayerController combines **physics-driven motion** with **manual input smoothing**:

- Horizontal velocity uses **acceleration/deceleration** curves to avoid abrupt starts/stops.
- Jumping includes **variable height** (short vs long press).
- Wall sliding limits fall speed; Wall Jump applies directional impulse.

- Dashing grants temporary gravity disable and burst motion.

Movement Hierarchy

```
Idle
 ├─ Move (walk/run)
 │    └─ Dash (horizontal burst)
 ├─ Jump (grounded)
 │    ├─ Double Jump
 │    └─ Wall Jump (if touching wall)
 └─ Fall → Land
```

Technical Parameters

Parameter	Description	Default
`moveSpeed`	Horizontal movement speed.	7
`jumpForce`	Vertical impulse applied on jump.	15
`fallMultiplier` / `lowJumpMultiplier`	Scales gravity for jump arcs.	2.5 / 2
`dashSpeed` / `dashDuration`	Dash distance & duration.	20 / 0.2s
`wallSlideSpeed`	Max fall velocity when sliding on wall.	2

Combat System

Combat is **close-range and timing-based**, with brief invulnerability windows after hits.

Attack Behavior

- Attack triggers a circular hitbox (`OverlapCircleAll`) from `attackPoint`.
- Enemies in range receive damage via `TakeDamage()`.
- The system supports combo expansion and animation syncing in future iterations.

Parameter	Description	Default
`attackDamage`	Damage per hit.	20
`attackRange`	Radius of hit detection.	0.5
`invincibilityTime`	Duration after being hit where damage is ignored.	1.0s

Abilities

Each ability is integrated via the **AbilityManager**, which checks unlocks from `GameManager`.

Ability	Description	Unlock Condition
Dash	Burst of movement; ignores gravity briefly.	Start
Double Jump	Adds second jump in air.	Start
Wall Jump	Jump off walls; enables wall slides.	Start
Ranged Pulse	Energy shot damaging enemies or activating devices.	After Boss 1
Super Dash	Continuous dash until collision.	After Boss 2

Abilities use `ScriptableObject` definitions (`AbilityData`) for stats, cooldown, and icon display.

Damage & Death Handling

- When damaged, the player enters a **blinking invincible state** via coroutine (`InvincibilityCoroutine`).
- On death:
  - Plays death SFX + fade out.
  - Triggers `GameManager.RespawnPlayer()` after delay.
  - Health resets to full.
- Damage updates both **UI** and **GameManager** for consistency across scenes.

Implementation Notes

- **Physics:** Rigidbody2D + OverlapBox for ground/wall detection.
- **Audio:** All events (jump, dash, attack, death) routed to `AudioManager`.
- **Animation:** Controlled by parameters: `Speed`, `VelocityY`, `IsGrounded`, `IsWallSliding`, `IsDashing`.
- **Debug Gizmos:** Visualizes ground, wall, and attack detection zones for tuning.

Technical Considerations

- **Performance:** Movement & input handled in FixedUpdate() for stable physics.
- **Extensibility:** Adding a new ability only requires creating a new `AbilityData` ScriptableObject.
- **Editor Tools:** Wall/jump/dash zones visualized via `OnDrawGizmosSelected()`.
- **Code Safety:** Uses `GameManager.Instance.HasAbility()` checks to gate ability usage.

Future Additions

Feature	Description
Charged Pulse	Hold button to charge and fire a larger energy shot.
Aerial Combo System	Expand melee combo tree for aerial chains.

III. Enemies

Overview

Enemies serve as the primary combat encounters and pacing elements in the game. Each type exhibits distinct movement and attack patterns that challenge the player's timing, positioning, and ability usage.

Enemy logic is powered by a **custom Behavior Tree (BT) system**, designed for readability, modularity, and easy expansion as new enemy types are introduced.

Enemy Types

Type	Behavior
Melee	Basic melee attacker; patrols short areas.
Ranged	Stationary ranged enemy; shoots in bursts.
Air	Flying enemies.
Guardian	Bigger enemies, can jump behind or use ranged attack to stagger.
Mini-Boss	Optional stronger enemy guarding upgrades.

Bosses

Boss	Theme / Focus	Reward
Boss 1	Tests basic combat and spacing.	Unlocks **Ranged Pulse**
Boss 2	Require pulse to hit while airborne.	Unlocks **Super Dash**
Boss 3	Final boss combining all previous mechanics.	Ends game

> Completing each area's optional Puzzle Trial before the boss weakens that boss (e.g., disables traps, reduces phase count, lowers health).

Behavior System

Enemy AI is structured using a **hand-implemented Behavior Tree**.

Unlike a Finite State Machine (FSM), which relies on explicit state transitions, the Behavior Tree continuously evaluates conditions in a hierarchical order. This structure keeps logic modular and scalable while avoiding "state explosion."

Core Components

Node Type	Description
Root	Entry point of the AI tree. Runs continuously.
Selector	Chooses the first successful child (used for decision-making).

Node Type	Description
Sequence	Executes children in order until one fails (used for action chains).
Decorator	Modifies child logic (e.g., repeat every frame, invert result).
Condition	Checks a specific boolean state (e.g., “PlayerDetected?”, “IsDead?”).
Action	Performs gameplay logic (e.g., move, attack, patrol).

Implementation Notes

- Each enemy will have a `BehaviorTree` component that updates once per frame.
- Nodes are implemented as small, reusable C# classes with a `Tick()` function returning **Success**, **Failure**, or **Running**.
- Conditions are lightweight checks (e.g., distance to player, current HP).
- Actions trigger animations, movement, or attacks using existing managers (e.g., `EnemyMovement`, `EnemyCombat`).
- Common subtrees (e.g., “Patrol and Chase,” “Attack and Die”) will be shared across multiple enemies.

Example: Melee Enemy

A basic ground unit that patrols until it detects the player, then transitions into chase and attack behaviors.

Behavior Summary

- **Patrol:** Walks along waypoints until player enters detection radius.
- **Chase:** Moves toward player until in attack range.
- **Attack:** Performs melee attack if close enough.
- **Die:** Plays death animation and disables logic when HP ≤ 0.

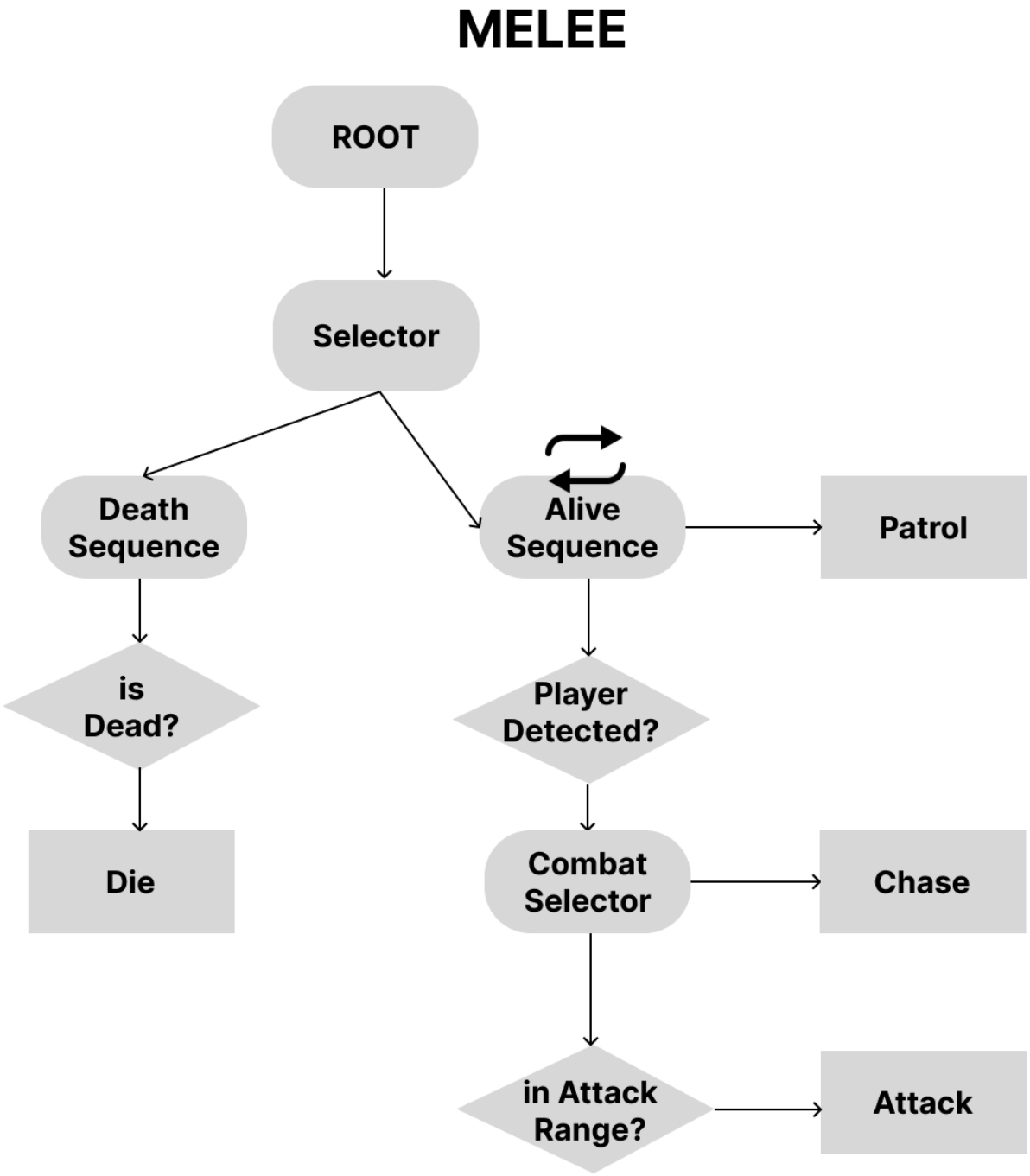
Behavior Tree Logic

```
Root
 └── Selector
      ├── Sequence (Death)
```

```
        │     ├── Condition: IsDead?
        │     └── Action: Die
        └── Decorator (Repeat)
              └── Sequence (Alive)
                    ├── Condition: PlayerDetected?
                    ├── Selector (Combat)
                    │     ├── Sequence (Attack)
                    │     │     ├── Condition: PlayerInRange?
                    │     │     └── Action: Attack
                    │     └── Action: Chase
                    └── Action: Patrol
```

Behavior Tree Diagram

https://www.figma.com/design/bCeQbGhwQDHR1APi5CykRM/Enemies-Behavior-Tree?t=TMqTHazItgCYI5AF-0



Future Enemy Types

Type	Behavior Notes
Ranged	Similar structure, adds projectile attack and cover-seeking logic.

Type	Behavior Notes
Flying	Uses modified movement actions (hover, dive) and larger detection radius.
Boss	Multi-phase tree: separate subtrees for phase transitions and ability rotations.

Technical Considerations

- **Performance:** Each `Tick()` call is lightweight; trees only process relevant subtrees each frame.
- **Reusability:** Common nodes (e.g., `MoveToTarget`, `CheckPlayerDistance`, `PlayAnimation`) shared across all enemies.
- **Debugging:** Trees will be visualized in editor via debug gizmos and optional in-scene node overlays.
- **Extensibility:** Adding new enemy behavior only requires new Condition or Action nodes, not rewriting logic.

IV. World

Overview

The world of *Echoes of the Depths* is a sprawling, interconnected underground kingdom once known as **Varelia**, now consumed by corruption seeping from the shattered Crystal Core.

Each region represents a fragment of the fallen civilization — from decaying royal halls to bioluminescent caverns — reflecting both the kingdom's downfall and the player's gradual reclamation of power.

The environment serves not only as a backdrop but as an extension of the gameplay: puzzles, traversal routes, and enemy behavior are all tied to the region's dominant element or ability focus.

World Structure

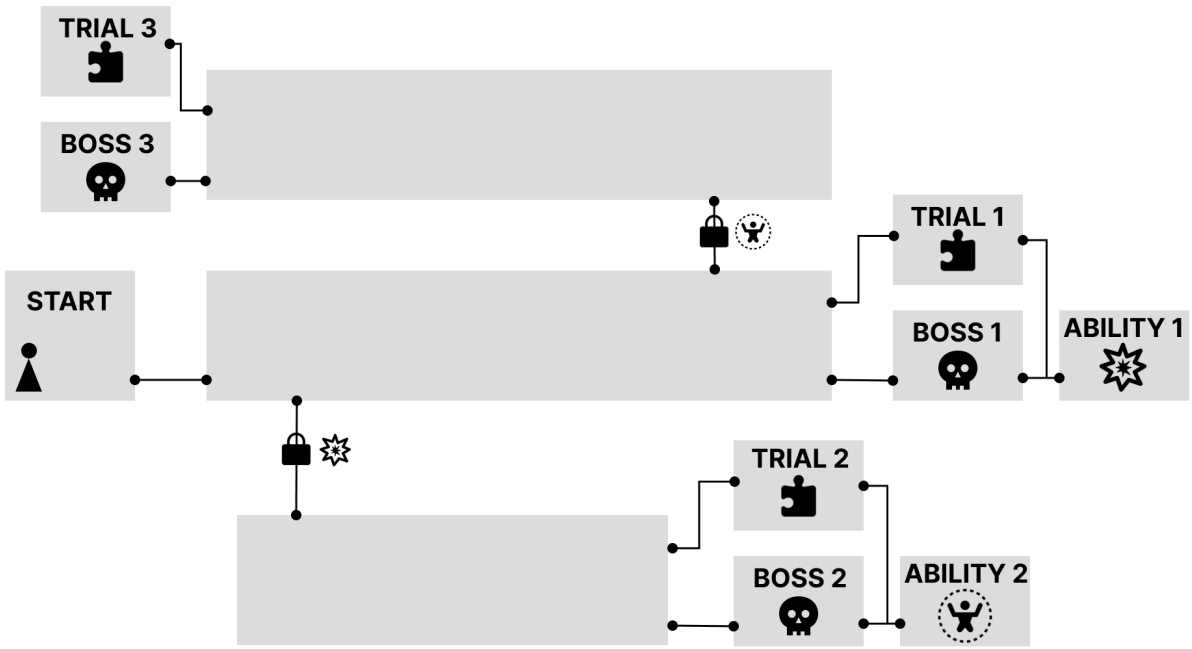
- **Interconnected Regions:** Each area connects organically, allowing seamless backtracking and shortcut discovery.
- **Ability-Gated Exploration:** Every region introduces one major ability essential for progression (e.g., Dash, Pulse).
- **Puzzle–Boss Synergy:** Each boss region contains an optional Puzzle Trial that, once completed, weakens the local boss.
- **Central Nexus (optional):** Later development may include a hub-like area where fast travel and NPC interaction occur.

Exploration Goals

- **Encourage curiosity:** Multiple branching paths reward backtracking.
- **Layered level design:** Each new ability reveals alternate routes in previously visited regions.
- **Environmental puzzles:** Switches, timed gates, moving platforms, and energy nodes are woven into traversal.
- **Hidden collectibles:** Relics grant passive buffs, lore fragments, or energy capacity upgrades.

Draft Layout

https://www.figma.com/design/V8q552I15jAHUBgRqFX4JX/Planned-Map-Layout?node-id=1-2&t=1iw27ALSDgRQdmH6-1



V. Architecture

Overview

The project will follow a **modular and data-driven Unity architecture**, prioritizing readability, scalability, and efficient iteration. Each gameplay system (player, abilities, puzzles, enemies) will be built around reusable components and ScriptableObjects to simplify balancing and future region additions.

Folder Structure

```
Assets/
 ├─ Scripts/
 │   ├─ Player/
 │   ├─ Abilities/
 │   ├─ Events/
 │   ├─ Data/
 │   ├─ Enemies/
 │   ├─ Managers/
```

```
│   └─ UI/
├─ Art/
│   ├─ Tilesets/
│   ├─ Characters/
│   └─ VFX/
├─ Audio/
│   ├─ Music/
│   └─ SFX/
├─ Input/
├─ Prefabs/
│   ├─ Abilities/
│   ├─ Player/
│   ├─ Enemies/
│   ├─ Environment/
│   ├─ UI/
│   └─ Puzzles/
├─ Scenes/
│   ├─ Region_01/
│   ├─ MainMenu/
│   ├─ _Manager/
│   └─ TestArena/
├─ ScriptableObjects/
│   ├─ AbilityData/
│   ├─ EnemyData/
│   └─ ItemData/
└─ Resources/
```

Scene Management

The game will use an **additive loading structure**:

- A persistent “Manager” scene holds global systems (GameManager, AudioManager, UIManager).

- Each region (e.g., *Region_01*) is loaded additively, allowing seamless backtracking and fast iteration.
- Scene transitions handled by `SceneLoader` utility class.

Core Systems and Classes

System	Class	Description	Key Variables
Game Manager	`GameManager`	Controls global state, handles pause, checkpoints, and ability progression.	`bool isPaused`, `List<string> unlockedAbilities`, `Transform respawnPoint`
Player Controller	`PlayerController`	Manages movement, animation, combat, and input.	`float moveSpeed`, `float jumpForce`, `int maxHealth`, `Rigidbody2D rb`, `Animator anim`
Ability Manager	`AbilityManager`	Tracks all unlocked abilities and activates their behavior.	`List<AbilityData> abilities`, `bool canDoubleJump`, `bool canDash`
Ability Data	`AbilityData` *(ScriptableObject)*	Defines ability stats and visuals.	`string abilityName`, `Sprite icon`, `float cooldown`, `AbilityType type`
Enemy AI	`EnemyController`	Base enemy behavior (patrol, chase, attack).	`float moveSpeed`, `int damage`, `float detectionRange`, `EnemyState state`
Enemy Data	`EnemyData` *(ScriptableObject)*	Stores per-enemy stats and loot drops.	`int health`, `int attack`, `float moveSpeed`, `GameObject dropItem`
UI Manager	`UIManager`	Controls health bars, ability icons, and fade transitions.	`Image healthBar`, `Image[] abilityIcons`, `CanvasGroup fadeGroup`
Audio Manager	`AudioManager`	Centralized control for background music and SFX.	`AudioSource bgmSource`, `AudioSource sfxSource`,

System	Class	Description	Key Variables
			`Dictionary<string, AudioClip> clips`
Scene Loader	`SceneLoader`	Handles additive scene loading and unloading.	`string currentRegion`, `AsyncOperation loadOp`

ScriptableObjects

Most variable game data (abilities, enemies, items) will use **ScriptableObjects** for flexibility.

Save & Checkpoint System

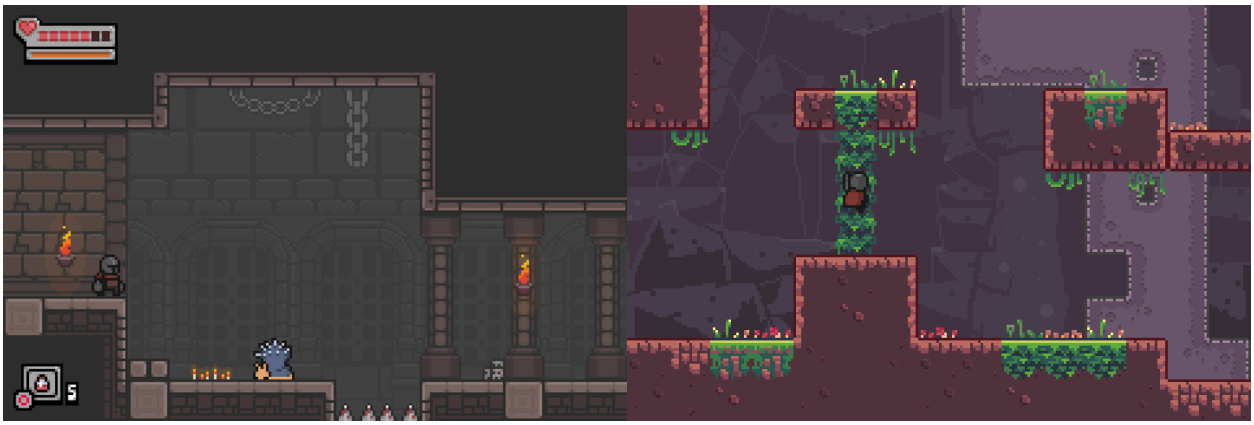
- Basic JSON-based save file storing:
  - Player position
  - Health
  - Unlocked abilities
  - Region progress
- Checkpoints trigger `GameManager.SaveGame()` upon interaction.
- Respawn handled via stored `respawnPoint`.

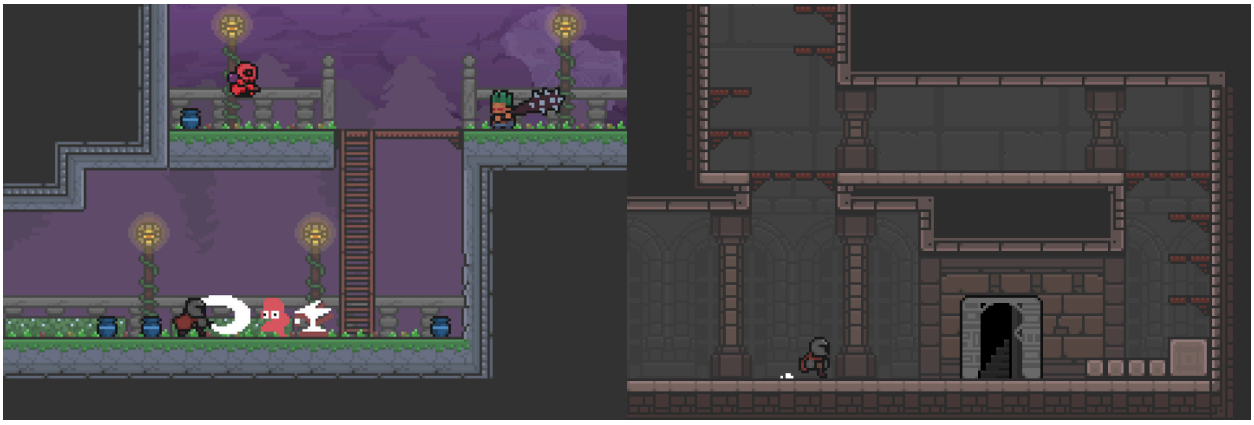
VI. Arts

Art Direction

- **Style:** Dark fantasy pixel art with muted tones and subtle lighting contrast.
- **Palette:** Emphasis on cool desaturated backgrounds contrasted by warm highlights (torches, energy crystals).
- **Inspiration:** *Blasphemous*, *Inmost*, *Axiom Verge*.
- **Asset Pack:** o-Lobster's *Another Metroidvania Asset Pack Vol. 1:* https://o-lobster.itch.io/another-metroidvania-asset-pack-vol-1

Environmental Samples





- **Sample 1 – Ruined Corridor:** Shows modular stone tiles, glowing torches, and environmental layering.
- **Sample 2 – Subterranean Cavern:** Blue-gray tones with dripping stalactites and reflective pools.
- **Sample 3 – Forgotten Shrine:** Warm gold and orange palette with ornate motifs.
- **Sample 4 – Surface Entrance:** Bright lighting with sharp contrast to emphasize descent into darkness.

Character Design Direction

- Silhouettes readable against dark backgrounds.

- Emphasis on visual clarity for attacks, abilities, and hit feedback.
- Idle and combat animations use subtle looping motion to create a “living ruin” feeling.